

The Splunk Playbook

A complete, detailed guide to Splunk — from data onboarding and SPL to dashboards, alerting, and detection engineering

From a raw log source to a tuned correlation search, this playbook builds the architecture knowledge and SPL fluency that SOC analysts and Splunk admins rely on every day.

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 5, 2026

Preface

Splunk is the platform most SOC analysts meet first, and for many it's the tool they spend the most working hours inside. This playbook goes far deeper than a general SIEM overview: it covers Splunk's actual architecture end to end, how data gets onboarded and parsed correctly in the first place, the full Search Processing Language rather than a handful of commands, knowledge objects and apps, dashboard and alert construction, and the administrative concepts (indexes, roles, licensing) that shape how a real deployment behaves.

The organizing idea is that most Splunk problems — a search that's too slow, a field that won't extract, an alert that never fires — trace back to a gap in one of these fundamentals rather than to SPL syntax itself. This book tries to close those gaps directly, with runnable examples throughout.

What this book covers

- Splunk architecture in depth: forwarders, indexers, search heads, deployment
- Data onboarding: inputs, props/transforms, parsing, and sourcetypes
- The full Search Processing Language, command by command
- Knowledge objects: fields, event types, tags, macros, lookups
- Building dashboards and visualizations
- Alerts, scheduled searches, and correlation logic
- Splunk apps and add-ons, including the Common Information Model
- Indexes, buckets, and data lifecycle management
- Users, roles, and access control
- Performance and search optimization
- Practical detection use cases in SPL
- Splunk administration essentials

Swastik Sagar

Contents

Preface	i
1 Splunk Architecture in Depth	1
1.1 The three logical tiers	1
1.2 Forwarders	1
1.3 Indexers	1
1.4 Search heads	1
1.5 Other core components	2
2 Data Onboarding	3
2.1 The data pipeline, in order	3
2.2 Input types	3
2.3 Parsing: props.conf and transforms.conf	3
2.4 Choosing sourcetypes and indexes	4
3 SPL: Search Fundamentals	5
3.1 Search pipeline mechanics	5
3.2 Search modes	5
3.3 Time modifiers	5
4 SPL: Core Commands	6
4.1 Transforming commands	6
4.2 Filtering and shaping commands	6
4.3 Field manipulation commands	6
4.4 Joining and correlating commands	7
5 Knowledge Objects	8
5.1 What knowledge objects are	8
5.2 Field extractions	8
5.3 Lookups	8
5.4 Macros	9

6	Dashboards and Visualizations	10
6.1	From search to panel	10
6.2	Choosing a visualization	10
6.3	Dashboard Studio vs. classic XML	10
6.4	Tokens and drilldowns	10
7	Alerts and Correlation	11
7.1	Anatomy of an alert	11
7.2	Alert actions	11
7.3	Scheduling and throttling	11
7.4	A worked correlation search	11
8	Apps and the Common Information Model	12
8.1	Apps vs. add-ons	12
8.2	The Common Information Model (CIM)	12
8.3	Notable apps worth knowing	12
9	Indexes and Data Lifecycle	13
9.1	Bucket aging	13
9.2	Index configuration basics	13
10	Users, Roles, and Access Control	14
10.1	Roles	14
10.2	Index-level access control	14
11	Performance and Search Optimization	15
11.1	Where search time actually goes	15
11.2	Practical optimization rules	15
11.3	Summary indexing	15
12	Practical Detection Use Cases	16
12.1	Brute force detection	16
12.2	Impossible travel	16
12.3	Rare process execution	16
12.4	DNS query volume spike	16
12.5	Data exfiltration by volume	16

13 Splunk Administration Essentials	17
13.1 The .conf file layering system	17
13.2 Useful admin CLI commands	17
13.3 Monitoring Splunk's own health	17

Splunk Architecture in Depth

1.1 The three logical tiers

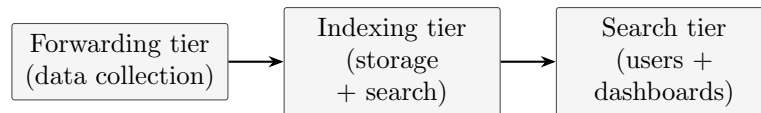


Figure 1.1. *Splunk’s three logical tiers can run on one box for a small lab, or scale to hundreds of indexers for an enterprise deployment.*

1.2 Forwarders

Type	Role
Universal Forwarder (UF)	Lightweight, minimal CPU footprint, forwards raw data without parsing
Heavy Forwarder (HF)	Full Splunk instance that can parse, filter, and route data before forwarding

Default choice

Universal Forwarders are the default choice for endpoints specifically because of their low resource footprint — Heavy Forwarders are reserved for cases that genuinely need pre-index parsing or filtering, such as routing different sourcetypes to different destinations.

1.3 Indexers

Indexers store data in **buckets** organized by age (hot, warm, cold, frozen) and build the index files that make searches fast. In a distributed deployment, an **indexer cluster** replicates data across multiple indexers for both search performance and data resilience.

1.4 Search heads

Search heads are where users and dashboards actually run searches; in larger deployments a **search head cluster** shares configuration and distributes search load across several search heads behind a single interface.

1.5 Other core components

Component	Role
Deployment Server	Pushes configuration/apps out to a fleet of forwarders
License Master	Tracks indexing volume against the license entitlement
Cluster Master	Coordinates indexer cluster replication and bucket fixups
Deployer	Distributes apps and configuration to a search head cluster

Data Onboarding

2.1 The data pipeline, in order



Figure 2.1. Every event Splunk ever indexes passes through these four stages, in this order, without exception.

2.2 Input types

Input type	Example
Files and directories	Monitoring a log directory for new/changed files
Network inputs	Listening on a TCP/UDP port, e.g. for syslog
Scripted inputs	Running a script periodically and indexing its output
Windows inputs	Event Logs, Registry, WMI, Active Directory
HTTP Event Collector (HEC)	Applications push JSON events directly over HTTPS

inputs.conf: monitoring a log file

```
[monitor:///var/log/auth.log] sourcetype = linux_secure index = auth
```

2.3 Parsing: props.conf and transforms.conf

props.conf controls how Splunk breaks a raw data stream into individual events and extracts their timestamps; **transforms.conf** defines reusable field-extraction and routing rules that props.conf can reference.

props.conf: basic event breaking and timestamp rules

```
[my_sourcetype] SHOULD_LINEMERGE = false LINE_BREAKER = ([\r\n]+) TIME_PREFIX =
timestamp= TIME_FORMAT = %Y-%m-%dT%H:%M:%S
```

Why parsing correctness matters

A sourcetype with the wrong `LINE_BREAKER` will merge multiple events into one, or split one event into several — both corrupt search results and statistics silently, often without any obvious error, so verifying event breaking on a new source is worth doing before it's

trusted for detection.

2.4 Choosing sourcetypes and indexes

- One **sourcetype** per distinct log format, so field extractions apply correctly
- Separate **indexes** by retention need, access-control boundary, or data volume, not just by source
- Avoid one giant index for everything — it makes both permissions and retention tuning impossible later

SPL: Search Fundamentals

3.1 Search pipeline mechanics

Every SPL search is a pipeline: an initial search term set finds raw events, and each subsequent `|`-separated command transforms the results of the command before it.

Anatomy of a search

```
index=firewall sourcetype=cisco_asa dest_port=445 | stats count by src_ip | where
count > 20 | sort -count
```

3.2 Search modes

Mode	Behavior
Fast	Minimal field discovery, fastest execution
Smart	Default; discovers fields for non-transforming searches, skips it when a transforming command is present
Verbose	Full field discovery and event data, slowest

3.3 Time modifiers

Common time range syntax

```
earliest=-24h latest=now earliest=-7d@d latest=@d earliest=@mon latest=now
earliest=0 latest=now
```

`@d/@mon` snap to the start of the day/month; combining a relative offset with a snap (`-7d@d`) is the standard way to express "the start of 7 days ago" rather than an exact 168-hour window.

Search efficiency basics

Filter by index, sourcetype, and time range first — before any other terms. Splunk narrows its search space as it reads these early terms, so front-loading the most selective filters is the single biggest lever on search speed.

SPL: Core Commands

4.1 Transforming commands

Command	Purpose
stats	Aggregate: count, sum, avg, dc (distinct count), grouped by field(s)
chart	Like stats, but shaped for visualization (one field becomes the x-axis)
timechart	Aggregate over time buckets, ideal for trend graphs
top / rare	Most/least common values of a field

stats examples

```
index=web | stats count, avg(response_time) as avg_rt by status_code
index=auth | stats dc(src_ip) as unique_sources by user
```

4.2 Filtering and shaping commands

Command	Purpose
where	Filter using a boolean expression, evaluated after fields exist
search	Additional filtering mid-pipeline, same syntax as the initial search
dedup	Remove duplicate events by specified field(s)
fields	Include or exclude specific fields from results
table	Display only named fields, in the given order
sort	Order results; prefix a field with - for descending
head / tail	Keep only the first/last N results

4.3 Field manipulation commands

Command	Purpose
eval	Create or transform a field with an expression
rex	Extract a new field using a regular expression
rename	Rename a field for display or downstream use
fillnull	Replace null/missing field values with a specified value
convert	Convert data types, e.g. string to number

eval and rex examples

```
index=web | eval status_category=if(status_code>=500, "error", "ok")
index=web | rex field=_raw "user=(?<username>\w+)" | stats count by username
```

4.4 Joining and correlating commands

Command	Purpose
<code>transaction</code>	Group related events into one transaction by shared field(s) and time proximity
<code>join</code>	SQL-style join against a second search's results
<code>append / appendcols</code>	Combine results from a second search, by row or by column
<code>lookup</code>	Enrich events with data from a lookup table

`transaction` vs. `stats`

`transaction` is expensive and best reserved for cases that genuinely need grouped, ordered events (like a full login-to-logout session). If you only need aggregate counts, `stats` is almost always faster and should be preferred.

Knowledge Objects

5.1 What knowledge objects are

Knowledge objects are reusable definitions layered on top of raw data — they don't change what's indexed, only how it's interpreted and presented at search time.

Object	Purpose
Fields & field extractions	Named values pulled from raw events, via regex or delimiters
Event types	Saved searches that tag matching events with a category name
Tags	Simple labels attached to specific field/value pairs
Macros	Reusable, parameterized snippets of SPL
Lookups	External tables (CSV or KV store) joined against events
Workflow actions	Right-click actions on a field, e.g. pivoting to another tool

5.2 Field extractions

Inline extraction with rex vs. a persistent extraction

```
-- ad hoc, this search only: index=web | rex field=_raw "user=(?<username>\w+)"
-- persistent: configured once via Settings > Fields > -- Field Extractions,
applies to every future search automatically
```

5.3 Lookups

A CSV lookup, defined then used

```
-- Settings > Lookups > Lookup table files: upload assets.csv -- Settings >
Lookups > Lookup definitions: name it "asset_lookup"
index=firewall | lookup asset_lookup ip AS dest_ip OUTPUT owner, criticality
```

Lookups for enrichment

A lookup mapping IP addresses to asset owners or criticality ratings turns a bare IP in an alert into something immediately actionable — this is one of the highest-value, lowest-effort improvements to make to any detection pipeline.

5.4 Macros

A parameterized macro

```
-- Settings > Advanced Search > Search Macros -- Name:  failed_logins_by(1) --  
Definition:  index=auth "Failed password" | stats count by $field$  
-- usage:   'failed_logins_by(src_ip)'
```

Dashboards and Visualizations

6.1 From search to panel

Every dashboard panel is, underneath, a saved SPL search feeding a visualization type — tables, line/bar/area charts, single-value panels, and maps are the most common choices.

A panel-ready timechart search

```
index=firewall action=blocked | timechart span=1h count by action
```

6.2 Choosing a visualization

Visualization	Best for
Table	Raw or aggregated rows, when exact values matter
Line/area chart	Trends over time
Bar/column chart	Comparing discrete categories
Single value	One key metric, often with a trend indicator
Choropleth/map	Geographic distribution

6.3 Dashboard Studio vs. classic XML

Newer Splunk versions default to **Dashboard Studio**, a more visual, JSON-backed builder with finer layout control; **classic Simple XML** dashboards remain fully supported and are still common in older deployments and many prebuilt apps.

6.4 Tokens and drilldowns

Dashboard **tokens** let one panel's selection (e.g. clicking a bar) filter what other panels on the same dashboard display — built through drilldown configuration rather than separate searches per panel, keeping dashboards interactive without duplicating query logic.

Alerts and Correlation

7.1 Anatomy of an alert

An alert is a saved search that runs on a schedule (or in real time) and triggers an action when its condition is met.

Trigger condition	Behavior
Number of results	Fires when the search returns more (or fewer) than a threshold
Number of hosts/sources	Fires based on distinct host/source count
Custom	Fires based on a custom SPL condition evaluated against results
Per-result	Evaluates the condition against each individual result, not the whole set

7.2 Alert actions

- Send email
- Run a script (e.g. a webhook or external API call)
- Add to Notable Events (Splunk Enterprise Security)
- Log an event back into Splunk itself, for meta-tracking

7.3 Scheduling and throttling

Cron-style scheduling

```
*/15 * * * * -- every 15 minutes 0 * * * * -- once per hour, on the hour
```

Throttling prevents duplicate alerts

Throttling suppresses repeat firings of the same alert for a matching field value within a configured window — essential for an alert condition that would otherwise fire again every scheduled run while the underlying issue persists, flooding the queue with duplicates.

7.4 A worked correlation search

Brute force followed by a successful login

```
index=auth (action=failure OR action=success) | transaction user maxspan=10m |
where mvcount(action)>1 AND searchmatch("failure") AND searchmatch("success")
```


Apps and the Common Information Model

8.1 Apps vs. add-ons

	Typically provides
App	A full user-facing experience: dashboards, saved searches, navigation
Add-on (TA)	Data collection and parsing for a specific source, normally no dashboards

A common pattern is installing a Technology Add-on (TA) on the forwarder tier to collect and normalize data, paired with a full app on the search tier for the actual dashboards and analysis.

8.2 The Common Information Model (CIM)

The CIM is a set of prebuilt field-naming standards — `src`, `dest`, `action`, `user`, and dozens more — that CIM-compliant add-ons map their raw fields onto, so a single search written against CIM field names works across many different vendors' data without modification.

Why CIM compliance matters

A detection written against `dest_port` instead of a vendor-specific field name keeps working when a firewall vendor changes, or when a second firewall brand gets added — CIM compliance is what makes detection content portable across a heterogeneous environment.

8.3 Notable apps worth knowing

App	Purpose
Splunk Enterprise Security (ES)	Full SIEM app: notable events, risk-based alerting, investigation workflows
Splunk IT Service Intelligence (ITSI)	IT operations monitoring and service health
Splunk Add-on Builder	Tooling for building custom TAs
Splunkbase TAs	Vendor-maintained add-ons for specific data sources (firewalls, cloud platforms, EDR)

Indexes and Data Lifecycle

9.1 Bucket aging

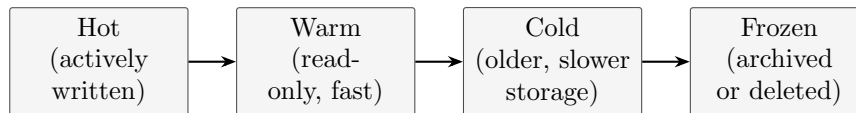


Figure 9.1. Every bucket ages through this path automatically based on size and time thresholds set per index.

9.2 Index configuration basics

indexes.conf: retention and sizing

```
[auth] homePath = $SPLUNK_DB/auth/db coldPath = $SPLUNK_DB/auth/colddb thawedPath
= $SPLUNK_DB/auth/thaweddb maxTotalDataSizeMB = 500000 frozenTimePeriodInSecs =
15552000
```

`frozenTimePeriodInSecs` controls retention by age (here, 180 days); `maxTotalDataSizeMB` caps retention by size — whichever limit is hit first triggers bucket freezing.

Frozen doesn't have to mean deleted

By default frozen data is deleted, but a `coldToFrozenDir` setting archives it to a specified path instead — useful for compliance retention requirements that outlast what's practical to keep searchable.

Users, Roles, and Access Control

10.1 Roles

Access in Splunk is governed by **roles**, which bundle capabilities (what a user can do) with index access (what data a user can search) and resource limits (search concurrency, CPU time).

Built-in role	Typical scope
admin	Full administrative access
power	Can create and share most knowledge objects, edit some settings
user	Can run searches and create personal knowledge objects only

10.2 Index-level access control

Restricting a role's **srchIndexesAllowed** list is how Splunk enforces that, say, an HR-data index is invisible to the general SOC analyst role — access control in Splunk is fundamentally index-based, which is why index design decisions made during onboarding directly shape what's later possible for permissions.

Design indexes with roles in mind

Splitting data into indexes by access-control boundary from the start is far easier than retroactively re-indexing data later to separate permission scopes — this is one of the most common regretted decisions in real deployments.

Performance and Search Optimization

11.1 Where search time actually goes

- **Search scope** — how much raw data has to be scanned before filtering narrows it down
- **Command cost** — some commands (`transaction`, `join`) are inherently expensive
- **Field extraction** — searching on an unindexed field means scanning raw text for every candidate event

11.2 Practical optimization rules

Rule	Why
Filter on index/sourcetype/time first	Narrows the search space before anything else runs
Use indexed fields when possible	Avoids scanning raw text for every event
Prefer <code>stats</code> over <code>transaction</code>	<code>stats</code> is far cheaper when grouped aggregates are enough
Use summary indexing for expensive recurring searches	Precomputes results once instead of recalculating on every run
Avoid leading wildcards	<code>*.example.com</code> can't use the term index efficiently; <code>example.com*</code> can

11.3 Summary indexing

A summary index stores the pre-aggregated output of an expensive search, computed on a schedule — dashboards and reports then query the small summary index instead of re-running the expensive search against raw data every time they load.

When summary indexing pays off

If a search is expensive AND queried repeatedly (a daily dashboard, for example), summary indexing usually pays for itself quickly. For a search run once during an ad hoc investigation, it's not worth the setup.

Practical Detection Use Cases

12.1 Brute force detection

SPL

```
index=auth "Failed password" | bin _time span=5m | stats count by src_ip, _time |  
where count > 15
```

12.2 Impossible travel

SPL, using a geo lookup

```
index=auth action=success | iplocation src_ip | stats earliest(_time) as  
first_seen, latest(_time) as last_seen, values(Country) as countries by user |  
where mvcount(countries) > 1
```

12.3 Rare process execution

SPL

```
index=endpoint sourcetype=sysmon EventCode=1 | stats count by host, process_name |  
where count < 3
```

12.4 DNS query volume spike

SPL

```
index=dns | bin _time span=1h | stats dc(query) as unique_queries by host, _time |  
where unique_queries > 500
```

12.5 Data exfiltration by volume

SPL

```
index=firewall direction=outbound | stats sum(bytes.out) as total_bytes by src_ip,  
dest_ip | sort -total_bytes | head 10
```

Splunk Administration Essentials

13.1 The .conf file layering system

Splunk configuration is layered across directories with a defined precedence order — system defaults, apps, and per-user settings can all define the same stanza, with the most specific applicable layer winning.

Typical config locations, in increasing precedence

```
$SPLUNK_HOME/etc/system/default/ $SPLUNK_HOME/etc/apps/<app>/default/
$SPLUNK_HOME/etc/apps/<app>/local/ $SPLUNK_HOME/etc/users/<user>/<app>/local/
```

Always edit local, never default

Changes belong in a **local** directory, never in **default** — upgrades overwrite **default** files, silently wiping any change made there.

13.2 Useful admin CLI commands

Splunk CLI basics

```
splunk status splunk restart splunk btool inputs list --debug splunk search
"index=_internal | head 10"
```

btool is especially useful for troubleshooting: it shows the final, merged configuration Splunk is actually using after layering, which is often not obvious from reading individual files.

13.3 Monitoring Splunk's own health

Internal indexes worth knowing

```
index=_internal -- Splunk's own operational logs index=_audit -- search and
configuration change history index=_introspection -- resource usage: CPU, memory,
disk I/O
```

license usage tracking

`index=_internal source=*license_usage.log*` is the standard starting point for diagnosing license violations or unexpectedly high daily indexing volume from a specific source.